

CS 354H Final Project Report

Goals: This project is a real-time renderer for a planet with an Earth-like atmosphere viewed from both surface and orbit, with two principal techniques: physically based atmospheric scattering (Rayleigh and Mie) and volumetric cloud ray marching with its own light scattering. The two techniques share the same raymarching infrastructure and produce a coherent visual experience. A player can fly close to the surface and see a blue sky with realistic sunsets, ascend through volumetric clouds, and view the planet from orbit with the proper planetshine. It should be noted that while the rendering techniques at play are rooted in real physics the atmosphere rendered is highly exaggerated for the sake of the viewing experience.

Atmospheric Scattering accounts for two physical phenomena. **Rayleigh scattering** describes how small molecules (particularly those smaller than the wavelength of visible light such as Nitrogen and Oxygen) scatter short-wavelength light more than long-wavelength light, producing the blue daytime sky and red sunsets. **Mie scattering** describes how larger aerosol particles scatter light wavelength-independently with a strong forward (anisotropic) bias, producing the bright halo around the sun. For each pixel a fullscreen fragment shader casts a view ray and integrates inscattered light along its path through the atmosphere shell. At each sample point, the shader computes the **optical depth** along a secondary ray to the sun (how much atmosphere light passes through to reach the sample), and accumulates light scaled by phase functions:

Rayleigh phase: $(3/16\pi)(1 + \cos^2\theta)$

Mie (Henyey-Greenstein) phase: $(3/8\pi)((1-g^2)(1+\cos^2\theta)) / ((2+g^2)(1+g^2-2g\cdot\cos\theta)^{3/2})$

Where θ is the angle between view and sun directions, and g is a constant determining the anisotropy of the Mie scattering. Atmospheric density falls off exponentially with altitude according to the Beer-Lambert Law, namely: $T = e^{(-\sum \beta \cdot \text{optical_depth})}$ where β is the scattering coefficient. For Rayleigh scattering these coefficients vary by channel whereas in Mie scattering they are constant.

Volumetric Clouds

Clouds are modeled as a 3D density field defined within a spherical shell. The density field combines a 3D Worley noise (for the base shape) with three octaves of Perlin (for detail erosion), pre-baked into an RGBA 3D texture at startup. The fragment shader raymarches through this shell using 64 fixed steps, sampling density and accumulating Beer's Law transmittance: $T_step = \exp(-\text{density} \cdot \text{stepSize} \cdot \text{extinction})$. Cloud lighting then uses three combined effects:

Henyey-Greenstein dual-lobe phase function: a 70/30 blend of forward ($g=0.8$) and back ($g=-0.5$) scattering, producing both silver lining and ambient cloud back-lighting.

Powder effect: $1 - \exp(-\text{density} \cdot 2)$ darkens cloud edges to fix the "cotton ball" look of pure single scattering.

Multi-step shadow march: For each cloud sample, a 16-step secondary ray marches toward the sun accumulating density. The resulting transmittance modulates direct sunlight allowing clouds to cast self-shadows.

A final planet shadow test in the shadow march correctly darkens night-side clouds: if the secondary ray to the sun intersects the planet, the cloud receives no direct sunlight. (Due to the ambient parameter clouds on the dark side of a planet will still be illuminated, as will the planet itself.)

Compositing: The final image is assembled in a composite pass. The pipeline runs in linear HDR throughout: $\text{final} = (\text{planet_albedo} \cdot \text{atmosphere_transmittance} + \text{atmosphere_color}) \cdot \text{cloud_transmittance} + \text{cloud_color} + \text{sun_disk} \cdot (\text{occlusion factors})$. ACES tonemapping followed by gamma correction maps HDR values to display-ready output.

Engineering: The renderer is implemented in TypeScript + WebGL2 using Vite as the build system. The codebase is organized into core abstractions (**Shader**, **Framebuffer**, **Mesh**, **FullscreenQuad**), rendering subsystems (**Camera**, **CubeSphere**, **NoiseVolume**), and shaders organized by purpose with shared `#include` files (`ray.glsl`, `reconstruct.glsl`, `scattering.glsl`, `constants.glsl`).

The render pipeline is a multi-pass forward renderer using `FrameBuffers` for intermediate storage:

1. **GBuffer pass:** rasterizes the planet (a cube-sphere mesh, resolution 100 per face, $\approx 60K$ vertices) writing albedo + depth to a framebuffer with `RGBA16F` color and `DEPTH_COMPONENT32F` depth.
2. **Atmosphere pass:** fullscreen quad samples GBuffer depth and raymarches the atmosphere shell, outputting `vec4(inscatter.rgb, transmittance)` to a `RGBA16F` target.
3. **Cloud pass:** identical structure to atmosphere, raymarches the cloud shell sampling the precomputed 3D noise volume.
4. **Composite pass:** combines GBuffer color, atmosphere, and clouds with the sun disk; applies ACES tonemap and gamma correction.

Nominally the planet is rendered at a scale of $1\text{unit} = 1\text{ km}$ (though even this scale is played with quite a bit). This is primarily done to avoid the headache of floating point precision that comes with realistically-sized planets. Atmospheric thickness is extremely exaggerated $\sim 30x$ to make the atmosphere visually prominent. Camera controls are planet-relative, see the README for more details.

The complete shader pipeline runs at full resolution, and runs reasonably (a.k.a. good enough) well on modern GPUs. Little optimization has been done, as the primary focus is on the base functionality.

Known Limitations and Bugs:

- **No procedural terrain.** The planet's surface is a smooth, single-color sphere. Heightmap-based terrain was planned but I ran out of time :(.
- **No floating origin / planet-scale precision.** Camera precision is preserved by scaling units to kilometers; this is sufficient for the demo but would not scale to multiple planets at solar-system distances, meaning we don't get Kerbal Space Program or even Outer Wilds.
- **Sharp edges on cloud tiling.** The 3D texture used for the planet is sampled too closely to cover the entire planet, producing a sharp edge where the texture loops.
- **Realistic shadows are not modeled.** The planet does not cast shadows on itself, the clouds, or the atmosphere. Meaning its dark side remains visible even when not lit, and ambient lighting keeps clouds visible even though their highlights disappear properly. This is most subtle with the atmosphere itself where proper shadowing would produce subtle pre-dawn glow effects, however these are not implemented producing too-bright twilights.